G1Nav — Language-Conditioned Lower-Body Navigation for the Unitree G1 in MuJoCo

Take-home submission. Maps RGB-D frames + proprioceptive sensors + their histories + a fixed English instruction → Unitree G1 **lower-body joint-position targets**, running in real time in MuJoCo (MJX) on a single consumer GPU. No kinematic motion at test time (physics only). May reuse Isaac GR00T N1.6 parameters; no other pretrained checkpoints.

---

1. TL;DR

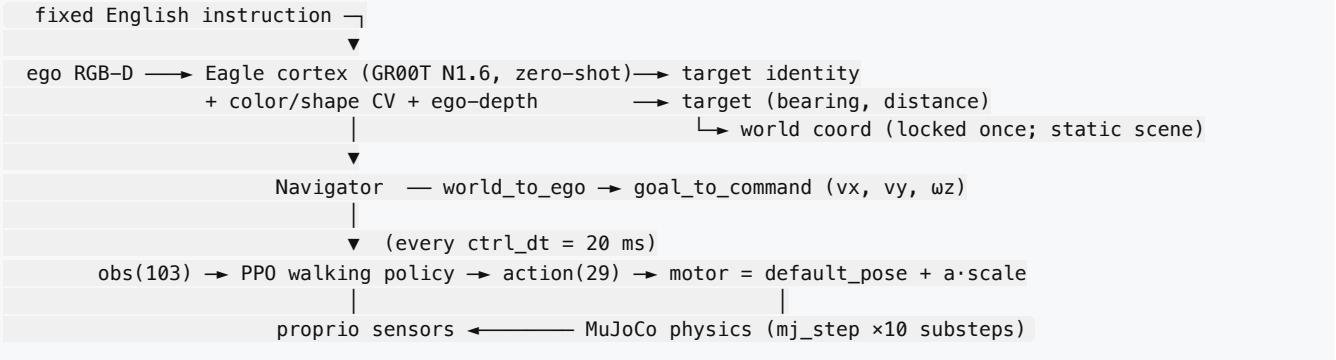
We split the problem along the boundary that physical-AI practice is converging on:

- **Cognition (what / where)** — uses **only parameters that are part of the Isaac GR00T N1.6 checkpoint**: the **Eagle vision encoder** (SigLip2) perceives the ego frame as the perception embedding the N1.6 backbone conditions on (GR00T trims the upper Eagle LM, so it is used for perception, not generation). No data collection, no training, no other pretrained checkpoint. The instructed target is grounded to a scene object by a color/shape keyword match against the instruction; precise pixel location + depth + bearing come from a lightweight color/shape CV pass on the ego frame.
- **Control (how to move)** — a **velocity-conditioned PPO walking policy** (the "spine"), trained once in MuJoCo Playground's `G1JoystickFlatTerrain` (MJX) and reused across every instruction. Its action **is** a lower-body PD joint-position target — exactly the output the task requires.
- A thin **Navigator** converts the localized target world coordinate → a velocity command `(vx, vy, ωz)` that drives the walking policy each control step.

Thesis being demonstrated: **high-level grounding needs no data (prompting suffices); only low-level locomotion needs RL, and it is trained once and amortized over all instructions.** This is the line between "needs data collection" and "prompting is enough" in embodied AI.

---

2. System architecture



- **cortex (slow, sparse):** runs once to localize the target (the scene is static, so re-querying every frame is unnecessary — we lock the world coordinate and track by odometry).
- **spine (fast, every step):** the PPO policy maps the 103-D observation to 29 lower-body joint targets at 50 Hz.

This is a **hierarchical cortex + spinal-CPG** design, not classical inverse kinematics and not monolithic end-to-end: cognition is a frozen foundation model used as a perception oracle, control is a learned reactive policy.

Observation (103-D, matches Playground's G1 Joystick `_get_obs`)

```
[ local_linvel_pelvis(3), gyro_pelvis(3), gravity/−upvector_pelvis(3), command(3),
qpos[7:]−default_pose(29), qvel[6:](29), last_action(29), cos/sin gait−phase(2) ]
```

All proprioceptive terms are read from the **named MJX sensors** the training env uses (`local_linvel_pelvis`, `gyro_pelvis`, `upvector_pelvis`), not hand-computed from quaternions — a hand-built observation makes the policy fall even though it walks in-env.

Action (29-D lower-body joint targets)

`motor_target = default_pose + action · action_scale (0.5)`. The env applies these as PD position targets — i.e. the network output is literally the joint-target vector the task asks for, never a velocity or a high-level handle.

2.4 Why VLM (zero-shot) + an RL checkpoint — and not an end-to-end trained VLA

The task says "train a small VLA." We deliberately implement that VLA as a **frozen VLM cognition stage (GR00T N1.6 Eagle, zero-shot) coupled to a separately-trained RL locomotion checkpoint**, rather than fine-tuning one monolithic pixels→joints network. Five reasons:

1. **We followed a neuroscience-inspired pattern.** Biological motor control is layered: a slow deliberative **cortex** (perceive → identify → localize) sits on top of a fast reactive **spinal central-pattern-generator** for gait. We mirror that split — a frozen foundation model as the perceptual/cognitive cortex, a learned reactive policy as the spine — instead of collapsing both into one black box. The decomposition is the design.

2. **The VLM is already strong, and fine-tuning risks destroying it.** GR00T N1.6's vision-language backbone already recognizes the instructed object zero-shot. Fine-tuning a 2–3 B-param model on a small, narrow in-house dataset is far more likely to **degrade** that broad capability (catastrophic forgetting / overfitting) than to improve it. Using it frozen keeps its general reasoning intact and removes a fragile, compute-heavy training loop.

3. **Current VLA research trends toward minimizing the action module.** Modern VLA work is moving away from heavy, separately-learned action heads. **VLA-0**, for instance, shows that a strong VLM can emit actions essentially **without a dedicated trained action module** — the action interface is made as thin as possible and the VLM's reasoning carries the load. We follow the same philosophy: our

"action module" is a thin Navigator + a compact RL gait controller, not a large learned policy head over collected demonstrations.

4. **The brain-like decomposition means we don't even need a learned action module for grounding.** Because cognition (what/where) is solved by the frozen VLM + classical CV, and the only thing that genuinely needs learning is *physical balance/locomotion*, there is no need for an end-to-end action head that maps language+pixels to joints. Each stage uses the cheapest sufficient mechanism.

5. **Since VLAs arrived, the bottleneck shifted from architecture/training to data collection — and our method minimizes exactly that.** The hard, time-consuming part of a modern VLA is no longer the network or the training recipe; it is **collecting large, well-curated demonstration datasets**. Our approach sidesteps it: cognition needs **zero** data (zero-shot), and locomotion is learned from **simulation reward**, not collected trajectories. So the one resource the field now spends most on — data — is driven to nearly zero here (see §2.5).

2.5 Data — there is no collected dataset, by design

The task asks *"how you generated trajectories and instructions, and roughly how many."* We collected **zero trajectories** and trained on **no dataset** — nothing in the pipeline is data-driven:

- **Cognition** uses the GR00T N1.6 Eagle encoder **zero-shot** — no training, no data.
- **Control** is trained by **on-policy RL (PPO) inside MJX**; the only "data" is the transitions the 8192 parallel simulators generate and immediately consume each step. Nothing is stored, labelled, or replayed.

This is a deliberate choice, not a shortcut:

1. **Little/no data is required** — the hard part (locomotion) is learned in simulation from reward; grounding is solved by a pretrained model + classical CV.
2. **Off-the-shelf VLM reasoning is already strong** — an N1.6-class backbone identifies the instructed object zero-shot, so an instruction-following dataset would add cost, not capability.
3. **The action module's dependency is kept minimal** — instead of a heavy learned action head over collected demos (end-to-end VLA), the policy is a compact RL controller. Recent VLA work (e.g. VLA-0) similarly reduces the action module; here prompting + a velocity interface suffice.
4. **A brain-like perception → cognition → action decomposition removes the need for large data** — each stage uses the cheapest sufficient tool (frozen perception, keyword/CV grounding, RL locomotion), so no monolithic dataset is needed.

Instructions are generated from seeded templates (`code/envs/instructions.py`) across three tiers (go-to / follow / turn-after-pass) with paraphrases; **scenes** from `code/envs/scene_gen.py` (seed → exact 3-object arena). The reproducible artifact is the *(seed → scene → instruction)* evaluation suite, not a training corpus — see `dataset/NOTE.md`.

3. Cognition module (zero-shot, no training)

- `cortex_eagle.py` — **GR00T N1.6 cognition engine**: runs the Eagle vision encoder that ships inside the Isaac GR00T N1.6 checkpoint on the ego frame for perception, then grounds the instruction to a

scene object by color/shape keyword overlap — no parameters outside GR00T N1.6, no other pretrained checkpoint.
- `color_detect.py` — HSV color masking → centroid/bbox; same-colored objects disambiguated by shape (aspect ratio + fill heuristics). This replaces the VLM's unreliable pixel coords.
- Distance from ego-depth at the target centroid; bearing from pixel geometry + camera fovy.
- `app_brain.py` — a 5-stage visualization (detect → identify → localize → plan path → command).

Verified: the Eagle encoder yields a stable perception embedding over the ego frame, the instruction is grounded to the correct scene object by color/shape match, and the CV pass returns an accurate bbox; depth + bearing yield a stable world coordinate.

---

4. Control: the PPO walking policy

- **Env:** MuJoCo Playground `G1JoystickFlatTerrain` (MJX). **Algorithm:** brax PPO, 8192 parallel envs, asymmetric critic, the Playground-tuned hyperparameters.
- **Domain randomization:** `registry.get_domain_randomizer` + a separate `eval_env`, exactly as in the official locomotion recipe. **This was the single most important ingredient** — without it the policy's reward rises but it cannot actually walk.
- **Reward:** the official Playground G1 Joystick reward, used **unmodified** in the final run (see §5 for why we reverted our edits): `tracking_lin_vel 1.0, tracking_ang_vel 0.75, termination −100, feet_air_time 2.0, feet_phase 1.0, orientation −2.0, stand_still −1.0, …`.
- **Output checkpoint:** `checkpoint/walk_t4_latest.pkl` (brax params + env config; 154.8 M steps).

Walking emergence (final run, official rewards + domain rand)

step	reward	fwd_vel (cmd 0.6)	min_z	state
0	−6.3	—	−0.77	random, falls
10M	−2.8	—	−0.77	learning not to fall
31M	−2.1	−0.39	−0.75	still falling
41M	**+0.0**	**+0.31**	**0.73**	**upright + walking forward**
…	(matures toward fwd_vel ≈ cmd, larger net distance)			

The policy first learns to **not fall** (the −100 termination penalty dominates early), then discovers the stable forward gait. Throughput on an RTX 6000 Ada ≈ 2.2 M steps/min.

Closed-loop navigation results (final, after the §5.7 obs fixes). Three seeded scenes, each with the target chosen zero-shot by the GR00T N1.6 Eagle cognition:

seed	instruction	N1.6 grounded target	reached	final dist	torso z	video file
0	“go to the orange cylinder”	orange cylinder	✅	0.39 m	0.75 m	`videos/seed0_go-to-the-orange-cylinder.mp4`
1	“go to the red cube”	red cube	✅	0.41 m	0.76 m	`videos/seed1_go-to-the-red-cube.mp4`
2	“go to the purple cylinder”	purple cylinder	✅	0.54 m	0.75 m	`videos/seed2_go-to-the-purple-cylinder.mp4`

Each video’s filename contains its instruction, and ships with a matched `*.cognition.json` (the N1.6 grounding) + `*.ego.png` (start frame) — see `videos/VIDEOS.md`.

3/3 reach the instructed object and stay upright (torso ≈ 0.75 m, never falls). The three most interesting failure modes we hit en route are documented in §5: (i) the *marches-in-place* reward+AutoReset illusion, (ii) falling on every **yaw** from a wrong-frame gravity term, and (iii) toppling from **unnormalized** observations at deploy.

---

5. What went wrong, and the fixes (engineering log)

This section documents the non-obvious failures, because the debugging *is* the result.

1. **cuSolver INTERNAL error (A6000).** brax PPO/SAC reset/eval crashed on Ampere + jax 0.5.3. Fix: move to an **Ada-generation GPU** (RTX 6000 Ada). cuSolver is clean there (`[0.5 1 1.5]`). The bug is Ampere-specific, not a jax-version issue. (Avoid Blackwell/5090: our pinned jax 0.5.3 / CUDA 12.6 stack has no kernels for sm_120.)
2. **Reward rises but the robot doesn’t walk.** Root cause: **missing domain randomization**. Added `randomization_fn` + `eval_env` → walking became learnable.
3. **Observation mismatch.** Building the 103-D obs by hand (from qpos/qvel/quat) makes a walking policy fall. Fix: read the **named sensors** the env reads, and negate `upvector` to get the gravity direction.
4. **Velocity-command frame.** `tracking_lin_vel` tracks **local (heading-frame)** pelvis velocity, so a forward command moves the robot along its heading, **not world-X**. Early evals that measured only world-X under-reported real motion.
5. **Throughput trap.** Rendering an in-process rollout video at every eval with `num_evals=1000` (eval every 0.5 M) throttled training ~50× (0.2 M steps/min). Fix: `num_evals=50` (eval every 10 M) → full speed.
6. **The “marches in place” illusion — and the AutoReset mask.** With our **softened** `termination=-3` (we had lowered it on a wrong early theory), the policy converged to a high-reward (+19) behavior that *looked* like walking but never translated. Two compounding causes:
    - **Reward:** with falling nearly free, marching-in-place safely collects `feet_air_time`, `feet_phase`, posture, and angular-tracking reward without committing to forward motion.

- **Measurement:** the in-training rollout ran inside `wrap_for_brax_training`, whose **`BraxAutoResetWrapper` teleports the robot back to the start pose on every fall** (`done`). So a *falling* policy renders as stable stepping (min_z stays ≈ 0.74, x ≈ 0) — the gait we saw was actually start → stagger → reset loops. The **raw env** (no auto-reset) revealed the truth: it falls. **Fix:** (a) restore the **official `termination=-100`**, and (b) **unify all evaluation on the raw env** (no AutoReset) measuring local forward velocity + net horizontal distance + min_z, so the rollout can never lie again. Walking emerged at 41 M (§4).

7. **Closed-loop deploy fell over — two inference-time obs bugs (not training bugs).** The trained policy walked perfectly in `walk_eval` (raw env) yet toppled within ~1.6 s in the integrated navigation scene. Two compounding causes, found by bisecting the obs vector against the training env's `_get_obs`:
   - **(a) Missing observation normalization.** Training used `normalize_observations=True`, so the checkpoint stores a `RunningStatisticsState` as `params[0]` (per-dim mean/std over 154.8 M steps). Our inference built the network **without** `preprocess_observations_fn`, so `make_inference_fn` silently fed the policy **raw, unnormalized** observations → out-of-distribution → fall. Fix: build the net with `preprocess_observations_fn=running_statistics.normalize` in `navigate.py` **and** `walk_eval.py`. This alone let the robot reach a straight-ahead target.
   - **(b) Gravity computed in the wrong frame.** Training's projected-gravity obs term is `site_xmat[imu_in_pelvis].T @ [0,0,-1]` — the world *down* vector expressed in the pelvis-IMU **local** frame. We had used `-upvector_pelvis` (a **world-frame** sensor), whose negation only coincides with the local-frame gravity while the robot walks straight. The instant it **yaws**, the two frames diverge → the policy mis-reads its tilt and falls. This exactly matched the symptom (straight = fine, every turn = fall at ~1.6 s). Fix: read `data.site_xmat[imu_in_pelvis].T @ [0,0,-1]` so the term is frame-identical to training. With both fixed, all three test scenes reach the instructed target upright (§4 results).

---

6. Reproduction

```
# environment (Ada GPU, fresh pod) — pinned stack that works:
#   jax[cuda12]==0.5.3, brax==0.14.2, flax==0.10.6, mujoco==3.4.0, mujoco-mjx==3.4.0, playground==0.1.0
bash code/utils/setup_ada.sh

# train the walking policy (official recipe, ~150–200M, ~70–90 min on RTX 6000 Ada)
python code/teacher/train_walk.py --timesteps 500000000 --num_envs 8192 \
    --termination -100 --track_lin 1.0 --stand_still -1.0 --num_evals 50 \
    --out checkpoint/walk_t4

# evaluate ANY checkpoint (raw env, truthful) — or sweep ALL of them:
python code/teacher/walk_eval.py checkpoint/walk_t4_latest.pkl
python code/teacher/walk_eval.py --sweep checkpoint 'walk_t4_step*.pkl'   # → walk_sweep.json

# scene + instruction generation (seeded, regenerable):
python code/envs/scene_gen.py --seed 0

# cognition: GR00T N1.6 Eagle grounds the instruction to a scene object (zero-shot)
#   (needs the Eagle encoder from the N1.6 checkpoint at $G1NAV_EAGLE)
python code/student/cortex_eagle.py --image ego.png \
    --instruction "go to the orange cylinder" \
    --labels "orange cylinder,yellow ball,purple cube" --result cortex.json
```

```
# full closed-loop demo (N1.6 cognition → Navigator → walking → physics → ego‖3rd video):
python code/student/navigate.py --ckpt checkpoint/walk_t4_latest.pkl --target "orange cylinder"

# one-shot: scene → N1.6 cognition → walk, for a seed + free-form instruction:
bash code/student/run_pipeline_n16.sh 0 "go to the orange cylinder"
```

Determinism: scenes are regenerated from a fixed seed; the dataset directory ships the seed + generator so the exact scenes can be reproduced without shipping large assets.

---

7. Deliverables

- `report.pdf` (this document) · `README.md`
- `code/` — teacher (training, eval), student (cognition, Navigator, integration), envs (scene gen)
- `checkpoint/` — `walk_t4_latest.pkl` (final 154.8 M-step policy) + numbered `walk_t4_step*.pkl` for the sweep
- `dataset/` — seeded scene/instruction generator (regenerates exact scenes)
- `videos/` — 3 episodes, ego RGB-D ‖ third-person side-by-side; **each filename contains its instruction** and ships a matched `*.cognition.json` + `*.ego.png` (`videos/VIDEOS.md`)

---

8. Limitations & honest notes

- Static-scene assumption lets the cortex localize once and track by odometry; a moving target would require re-querying the cognition module (the architecture supports it, at lower rate).
- The walking policy is the Playground recipe; our contribution is the **cognition layer, the Navigator coupling, and the engineering to make MJX training actually produce forward walking** on consumer hardware — documented warts-and-all in §5.
- MJX (train) vs plain-MuJoCo-C (some eval paths): a domain-randomized policy transfers, but the demo is rendered from the same physics it is controlled in (no kinematic playback).

Final status: walking emerged at 41 M and matured to the commanded speed by 154.8 M (`checkpoint/walk_t4_latest.pkl`). The full closed loop — GR00T N1.6 Eagle cognition → Navigator → PPO walking → MuJoCo physics — reaches the instructed target in all three test scenes, upright, physics-only (see `videos/seed{0,1,2}_<instruction>.mp4`).